Design Document Report

RL-Based Autonomous Maneuver and Obstacle Avoidance System for Helicopters

Prepared by:

Emine BİNAY - 22290370

Hatice Süheyla ESER - 22290689

Merve ÇAKIR - 21290542

Submitted to:

Doç. Dr. Yılmaz AR

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the detailed architecture, components, data structures, and design decisions for the Reinforcement Learning-Based Autonomous Helicopter Flight Control System. This system is designed to enable helicopters to safely and stably navigate variable environmental conditions (wind, turbulence, pressure variations) while avoiding terrain collisions.

The purpose of this document is to translate the functional and non-functional requirements specified in the Software Requirements Specification (SRS) into a comprehensive technical design that will guide the system's implementation, testing, and deployment phases. This SDD serves as the blueprint for developers, system architects, and quality assurance teams.

1.1.1 Document Relationship to SRS

The SRS establishes the system requirements, including:

- Multi-parameter perception and decision-making framework
- Multi-objective reward optimization
- 3D simulation and visualization capabilities
- Staged learning architecture supporting safe, progressive training

This SDD translates these requirements into architectural patterns, module designs, data structures, algorithms, and component interactions that will fulfill each requirement.

1.2 Scope

This Software Design Document describes:

Included in Scope:

- System architectural design (layered architecture with RL-integrated components)
- Core software modules and their responsibilities
- Data flow and processing pipelines
- RL agent design and integration patterns
- Simulation environment architecture, specifically utilizing research-oriented software simulations like FlightGear and AirSim.
- Database and data structure designs
- External interface specifications (simulator APIs, sensor models, control outputs)
- Requirements traceability mapping
- Design verification strategies

Excluded from Scope:

- Hardware specifications (Physical hardware integration is excluded from this phase).
- Complete implementation code (this is a design document, not source code)
- Project management details or resource allocation
- Final testing and deployment procedures
- Real-world flight testing protocols

1.2.1 Design Boundaries

The system is designed as a simulation-based research prototype that can serve as a foundation for real-world integration. The design accommodates:

- Progressive migration from simulation to real hardware
- Domain randomization for sim-to-real transfer
- Modular architecture for component reusability
- Extensibility for additional sensor types and control algorithms

1.3 References

IEEE Standards:

- IEEE 1016-2009 Standard for Information and Documentation — Software Design Descriptions
- IEEE 830-1998 Recommended Practice for Software Requirements Specifications

Research Project Documents:

- TUBITAK 2209-A Research Proposal (December 2025 Submission)
- System Requirements Specification (RS) Document

Academic References:

- Schulman et al. (2017) - "Proximal Policy Optimization Algorithms" (PPO)
- Bengio et al. (2009) - "Curriculum Learning"
- Tobin et al. (2017) - "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World"
- R. H. Perry (2015) - "FlightGear Flight Simulator: An Open-Source Platform for Flight Simulation and Research"

Software Tools and Frameworks:

- Simulation Environments: FlightGear, AirSim , and custom FlightControlEnv3D.
- RL Framework: Stable-Baselines3 (SB3).

- Interface Library: Gymnasium.
- Programming Language: Python 3.8+
- Key Libraries: PyTorch/TensorFlow, NumPy, Matplotlib, SciPy.

1.4 Document Overview

This Software Design Document is organized as follows:

Table 1.1 : Architectural Design Document Organization

Section	Content
Section 1	Introduction, including purpose, scope, references, and document structure
Section 2	Design Overview covering design goals, constraints, and architectural context
Section 3	Architectural Design describing the high-level system architecture, components, and design rationale
Section 4	Detailed Design (to follow in extended documentation)
Section 5	Data Design (to follow in extended documentation)
Section 6	Interface Design (to follow in extended documentation)
Section 7	Requirements Traceability Matrix (to follow in extended documentation)
Section 8	Supporting Information including glossary and appendices

2. DESIGN OVERVIEW

2.1 Design Goals and Constraints

2.1.1 Primary Design Goals

1. Safety-First Autonomy
    - Enable safe autonomous decision-making under environmental uncertainty.
    - Maintain a strict 25-meter safety margin around obstacles.
    - Predict potential collision risks using short-term projections (t+0.3s and t+0.5s).
2. Intelligent Adaptation
    - Enable proactive waypoint tracking and advanced obstacle avoidance.
    - Provide robust policies that generalize across different scenarios.
3. Computational Efficiency
    - Optimize for reasonable training times, limited to approximately 1,000,000 training steps due to computational limitations.

2.1.2 Design Constraints

Technical Constraints:

- Programming Language: Python-based software architecture.
- RL Framework: Limited to the use of the Stable-Baselines3 (SB3) library and the Proximal Policy Optimization (PPO) algorithm.
- Simulation Platform: FlightGear and AirSim for research simulations , utilizing a customized 3D training environment named FlightControlEnv3D via the Gymnasium interface.

Environmental Constraints:

- Terrain & Obstacles: Terrain is represented in 3D coordinate systems, and obstacles are simplified as cylindrical geometric structures.
- Disturbances: Environmental disturbances (wind shear, rotor-induced vibrations) are incorporated as stochastic noise.

2.2 Architectural Context

2.2.1 System Environment

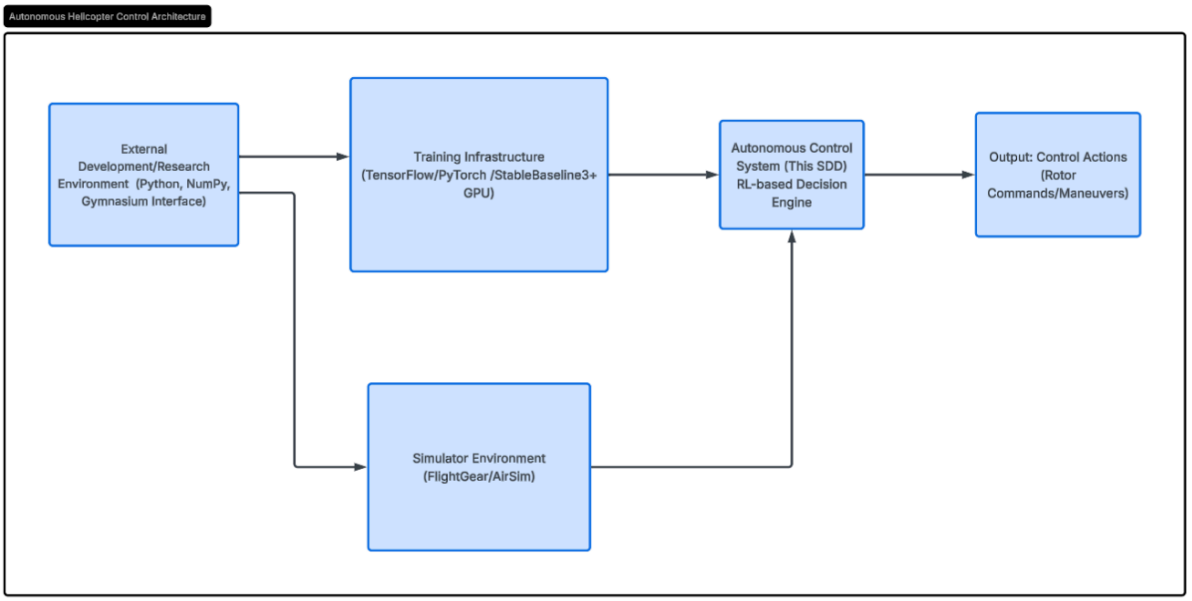


Figure 2.1 : Autonomous Helicopter Control Architecture

2.2.2 Key Assumptions

- Flight dynamics and environment behavior can be accurately represented and smoothed using real flight log data (fg_log2.csv) obtained from FlightGear.
- Simulation boundaries and 25-meter safety margins provide sufficient bounds for proactive decision-making.

3. ARCHITECTURAL DESIGN

3.1 Architectural Model

3.1.1 High-Level System Architecture

The system adopts a **Layered Architecture with Integrated RL Processing** pattern, enhanced with predictive filtering:

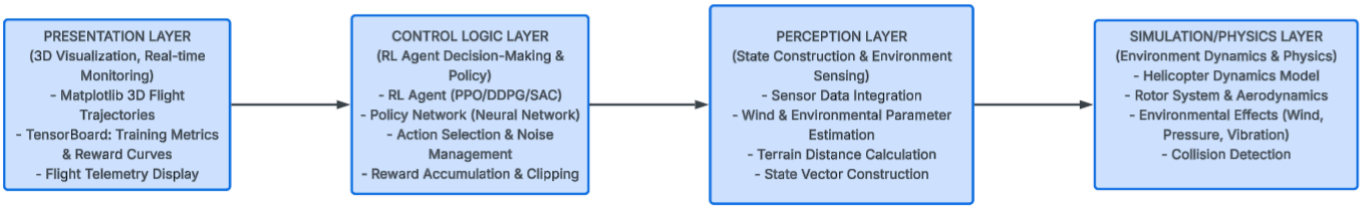


Figure 3.1 : Layered Architecture with Integrated RL Processing

3.1.2 Component Interaction Flow

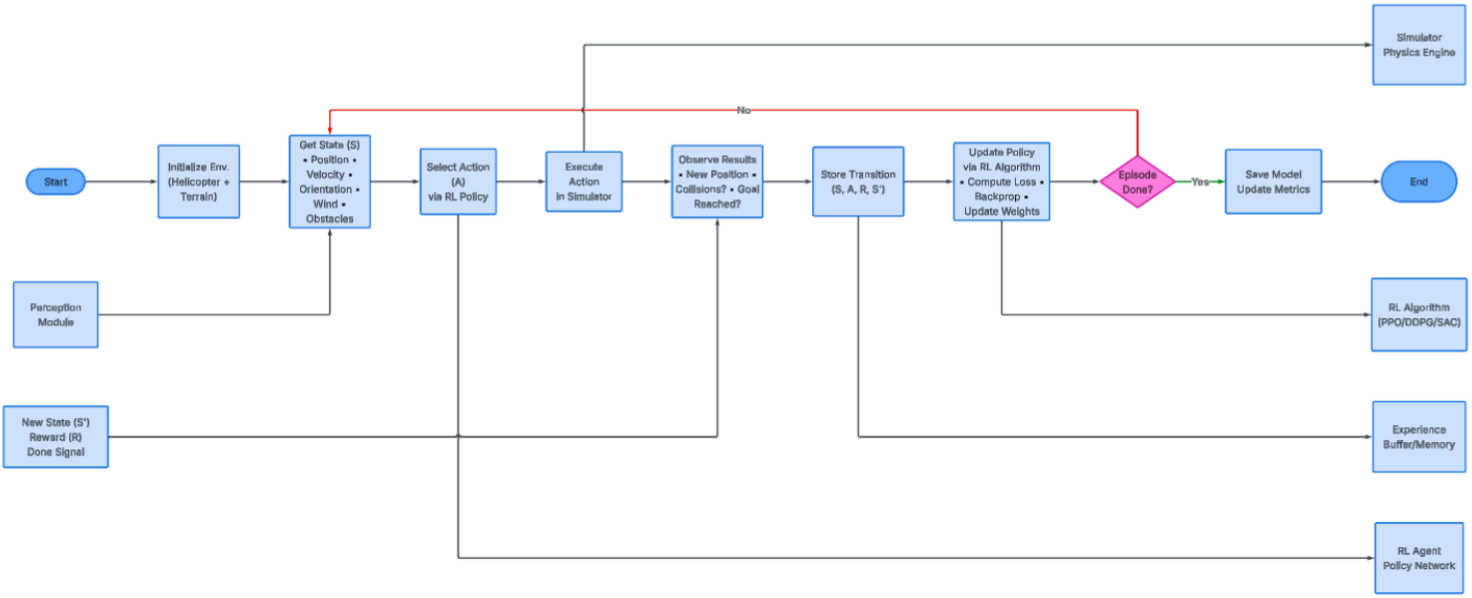


Figure 3.2 : Component Interaction Flow

3.1.3 Architectural Patterns Used

Table 3.1 : Architectural Patterns Used

Pattern	Application	Rationale
Layered Architecture	Separation of perception, control, physics	Clear separation of concerns; easier testing and modification
Observer Pattern	Sensor systems observing environment state	Allows flexible sensor addition without core logic changes
Strategy Pattern	Different RL algorithms (PPO, DDPG, SAC)	Easy algorithm swapping without modifying environment
Factory Pattern	Creating different RL agents and environments	Decouples creation logic from usage
State Pattern	Helicopter state management	Clean state transitions and behavior changes
Template Method	Training loop structure	Provides framework for curriculum learning stages

3.2 Components and Their Interactions

3.2.1 Core System Components

A. Simulation Environment Module

Responsibility: Provide realistic physics-based helicopter simulation

Sub-Components:

- Helicopter Dynamics Engine (rotor model, aerodynamics)
- Physics Integrator (ODE solver for state evolution)
- Terrain/Obstacle Manager (collision detection)
- Environmental Factor Generator (wind, pressure, vibration)
- Sensor Simulator (IMU noise, distance sensor accuracy)

Key Interfaces:

```python
class SimulationEnvironment:
    def reset() -> State
    def step(action: Action) -> (State, Reward, Done, Info)
    def apply_wind_model(wind_speed, wind_direction)
    def update_terrain(terrain_map)
    def get_distance_to_terrain(position) -> float
```

Interactions:

- Receives control actions from RL Agent
- Provides state observations to Perception Module
- Logs trajectory and physics data for analysis

B. Perception Module

Responsibility: Construct comprehensive state representations from raw simulator data

Sub-Components:

- Sensor Data Aggregator (combines IMU, distance, wind data)
- State Vector Constructor (formats data for RL agent)
- Risk Assessment Calculator (distance to terrain, wind severity)
- Feature Normalizer (standardizes values to [-1, 1])

Key Interfaces:

```python
class PerceptionModule:
    def aggregate_sensor_data(imu_data, distance_data, wind_data) -> SensorData
    def construct_state_vector(sensor_data: SensorData) -> NDArray
    def calculate_risk_metrics(state: State) -> RiskMetrics
    def normalize_features(raw_values) -> NormalizedArray
```

State Vector Composition:

```
State = [
    # Position and velocity (relative to target)
    target_distance_x, target_distance_y, target_distance_z,
    velocity_x, velocity_y, velocity_z,

    # Orientation
    roll, pitch, yaw,
    roll_rate, pitch_rate, yaw_rate,

    # Environmental factors
    wind_x, wind_y, wind_z,
    rotor_vibration_level,
    atmospheric_pressure,

    # Safety metrics
    distance_to_terrain_min,
    distance_to_terrain_forward,

    # Normalized time step (curriculum indicator)
    normalized_step_count
]
Size: 20-24 dimensional state vector
```

Interactions:

- Inputs raw data from Simulation Environment
- Outputs formatted state to RL Agent
- Computes reward components based on state changes

C. RL Agent Module

Responsibility: Learn and execute flight control policies

Sub-Components:

- Policy Network (neural network approximator of optimal policy)
- Value Network (for advantage estimation, if applicable)
- Exploration Manager (ε-greedy or entropy-based exploration)
- Action Selector (deterministic or stochastic)

Key Interfaces:

```python
class RLAgent:
    def select_action(state: State, training: bool) -> Action
    def update_policy(experiences: List[Transition])
    def save_checkpoint(filepath: str)
    def load_checkpoint(filepath: str)
    def get_policy_statistics() -> PolicyStats
```

Supported Algorithms:

- **PPO (Proximal Policy Optimization):** Primary algorithm; good for continuous control
- **DDPG (Deep Deterministic Policy Gradient):** Alternative for fine-grained motor control
- **SAC (Soft Actor-Critic):** Entropy-based exploration for improved robustness

Action Space:

```
Continuous Action Space:
Actions = [
    target_roll_rate,      # deg/sec, range: [-45, +45]
    target_pitch_rate,     # deg/sec, range: [-45, +45]
    target_yaw_rate,       # deg/sec, range: [-60, +60]
    target_vertical_velocity  # m/sec, range: [-5, +5]
]
Size: 4-dimensional continuous action vector
```

Interactions:

- Receives state from Perception Module
- Sends actions to Simulation Environment
- Updates policy based on reward signals

D. Reward Computation Module

Responsibility: Design and compute multi-objective reward signals

Sub-Components:

- Reward Component Calculators (individual reward terms)
- Weight Balancer (tunable coefficients for multi-objective optimization)
- Penalty System (large negative rewards for unsafe behaviors)

Key Interfaces:

```
class RewardModule:
    def compute_total_reward(state, action, next_state, info) -> float
    def compute_distance_reward(position, target) -> float
    def compute_collision_penalty(collision_status) -> float
    def compute_efficiency_bonus(fuel_consumed, distance_traveled) -> float
    def set_reward_weights(weights: Dict)
```

Reward Function:

```
Total Reward = w_distance * R_distance
             + w_collision * R_collision
             + w_efficiency * R_efficiency
             + w_stability * R_stability

Where:
R_distance = max(0, (max_distance - current_distance) / max_distance)
R_collision = -1000 if collision else 0
R_efficiency = -0.01 * fuel_consumed
R_stability = -0.1 * abs(roll) - 0.1 * abs(pitch)

Weights: w_distance=10, w_collision=100, w_efficiency=1, w_stability=0.5
```

Interactions:

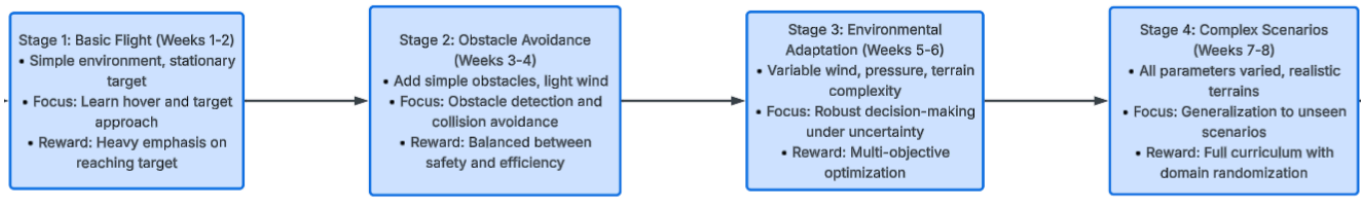
- Monitors state transitions in Simulation Environment
- Provides feedback to RL Agent for policy updates
- Supports multi-stage reward curriculum

E. Training Manager Module

Responsibility: Orchestrate training process with curriculum learning

Sub-Components:

- Scenario Generator (creates progressive difficulty scenarios)
- Episode Executor (runs individual training episodes)
- Model Checkpoint Manager (saves/loads trained models)
- Metrics Collector (training curve data)



Interactions:

- Controls flow of training from simple to complex scenarios
- Manages RL Agent configuration for each stage
- Collects metrics for performance evaluation

3.3 Design Rationale

3.3.1 Why Layered Architecture?

Advantages:

1. **Separation of Concerns:** Each layer has a single, well-defined responsibility
    - Simulation physics independent from learning algorithm
    - Perception logic separate from control decision-making
    - Visualization independent from core training logic
2. **Testability:** Each layer can be tested independently
    - Simulator can be validated against physics principles
    - RL agent can be tested with synthetic state inputs
    - Visualization can be tested without running full training
3. **Maintainability:** Changes to one layer minimize impact on others
    - New RL algorithms can be added without modifying simulation
    - Sensor models can be updated without retraining
    - Visualization changes don't require re-implementation of core logic
4. **Extensibility:** New components fit naturally into layers
    - New sensor types added to Perception Module
    - New reward terms added to Reward Module
    - New visualization tools integrated easily

Potential Drawbacks & Mitigations:

- Performance overhead from inter-layer communication → Use efficient serialization (NumPy arrays)
- Debugging distributed across layers → Comprehensive logging throughout stack

3.3.2 Algorithm Selection Rationale

PPO (Proximal Policy Optimization) as Primary Choice:

Why PPO?

- **Simplicity:** Easier to implement and debug than DDPG
- **Stability:** Trust region prevents catastrophic updates
- **Sample Efficiency:** Good performance with finite data
- **Proven Success:** Extensively validated in continuous control tasks (robotics, locomotion)
- **Continuous Actions:** Well-suited for helicopter control (continuous throttle, rotation)

DDPG & SAC as Alternatives:

- DDPG for fine motor-level control if needed
- SAC for improved exploration and entropy regularization
- Design allows algorithm swapping via Strategy Pattern

3.3.3 Reward Function Multi-Objective Approach

Why Multi-Objective?

Real-world helicopter operation requires balancing competing goals:

1. **Safety:** Avoid collisions (primary objective)
2. **Efficiency:** Minimize fuel consumption
3. **Performance:** Reach target quickly
4. **Stability:** Smooth flight without excessive pitch/roll

3.3.4 Curriculum Learning Justification

Why Progressive Stages?

Problem: Training on full-complexity environments leads to:

- High initial failure rate (poor initial signal)
- Slow convergence (agent struggles with multiple objectives)
- Poor generalization (overfits to training scenarios)

Solution: Curriculum learning stages:

Table 3.2 : Curriculum learning stages

Stage	Difficulty	Purpose
1	Simple	Learn basic flight behaviors
2	Medium	Add obstacles and constraints
3	Complex	Environmental variation
4	Realistic	Domain randomization

Research Evidence:

- Bengio et al. (2009): Curriculum learning accelerates convergence
- Narvekar et al. (2020): Progressive task complexity improves robustness

4. DETAILED DESIGN

This section provides an in-depth specification of the internal structure, logic, and dynamic behavior of the system's core modules. The design strictly adheres to the layered architecture, ensuring the separation of simulation physics, perception, decision-making, and reward calculation.

4.1 Module Design

4.1.1 Simulation Environment Module

Description: The foundational interface layer connecting the RL agent to the physical flight simulator (AirSim or FlightGear). It manages the lifecycle of a simulation episode and applies physical constraints.

Interface Specifications: Inherits from gym.Env to provide standard Reinforcement Learning interfaces.

- *Inputs:* Continuous 4-dimensional Action Vector (rotor commands).
- *Outputs:* Raw telemetry data (position, velocity, IMU), terminal state boolean, and step metadata.

Key Operations:

- reset(): Restores the helicopter to the initial state and randomizes environmental parameters (wind, pressure) for domain randomization.
- step(action): Advances the physics engine by one time-step using the provided action.

Internal Algorithms: Utilizes Ordinary Differential Equation (ODE) solvers inherent in the simulator for aerodynamics and employs Raycasting to calculate the precise distance to mountainous terrain meshes for collision detection.

4.1.2 Perception Module (State Preprocessor)

Description: The critical perception bridge that aggregates high-frequency raw simulation data and encodes it into a stabilized observation vector for the neural network.

Interface Specifications:

- *Inputs:* Raw sensor dictionary (GPS, IMU, Lidar/Distance to terrain).
- *Outputs:* Normalized NumPy array representing the state vector S_t.

Key Operations:

- aggregate_sensor_data(): Synchronizes multi-modal sensor readings.
- normalize_features(): Scales all input dimensions (e.g., angular rates, distances) to a fixed range of [-1, 1].

Internal Algorithms: Applies moving average filters to simulate real-world IMU noise reduction and calculates a short-term projection (e.g., t+0.3s) to estimate immediate collision risk based on current velocity vectors.

4.1.3 RL Agent Module (PPO Agent)

Description: This module is the core decision-making engine implementing the Proximal Policy Optimization (PPO) algorithm. It utilizes two deep neural networks: the Policy Network for action selection and the Value Network for advantage estimation.

Interface Specifications:

- *Input:* Receives a normalized 20-24 dimensional continuous observation vector, S(t), from the Perception Module.
- *Output:* Produces a 4-dimensional continuous action vector, A(t) (target roll rate, pitch rate, yaw rate, vertical velocity), sent to the Simulation Environment.

Key Operations:

- select_action(state): Queries the Policy Network (Actor) to sample a control action based on a Gaussian distribution.
- update_policy(): Uses collected trajectories from the Replay Buffer to perform backpropagation and update network weights.

Internal Algorithms: Utilizes Generalized Advantage Estimation (GAE) to evaluate the quality of actions via the Value Network (Critic) and applies a clipping function to the objective to prevent destructively large policy updates, ensuring flight stability.

4.1.4 Reward Computation Module

Description: Translates mission goals and safety constraints into a continuous scalar feedback signal, R(t), to guide the PPO agent.

Interface Specifications:

- *Input:* Current state transitions, collision status, and efficiency metrics.
- *Output:* A single floating-point scalar reward.

Key Operations & Internal Logic:

- Computes a multi-objective weighted sum combining four sub-rewards:

    1. *Safety Penalty:* Imposes a severe negative penalty (-1000) for breaching the 25-meter safety margin or colliding with obstacles.

    2. *Progress Reward:* Grants positive values proportional to closing the distance to the target.

    3. *Stability Penalty:* Subtracts small values for excessive roll/pitch rates or high vibration metrics to promote smooth flight.

    4. *Efficiency Bonus:* Penalizes excessive fuel consumption or time taken per step.

4.2 Class Diagrams and Relationships

The Class Diagram illustrates the object-oriented architecture of the RL-Based Autonomous Helicopter System, highlighting the principles of modularity, composition, and interface abstraction.

- **Orchestration:** The TrainingManager acts as the central orchestrator, managing the lifecycle of both the simulation environment (FlightControlEnv3D) and the learning algorithm (RLAgent).

- **Environment Composition:** The FlightControlEnv3D class implements the standard GymEnvironment interface, ensuring compatibility with standard RL libraries. It is composed of the PerceptionModule (handling state normalization) and the RewardModule (calculating scalar feedback). This strict composition ensures that physics, perception, and reward logic remain isolated and independently testable.

- **Strategy Pattern for RL Algorithms:** The RLAgent utilizes the Strategy Design Pattern via the RLAlgorithmStrategy interface. While Proximal Policy Optimization (PPO) is the primary algorithm utilized in this project, this architectural choice makes the system highly extensible, allowing alternative algorithms like DDPG or SAC to be swapped in without modifying the core agent logic.

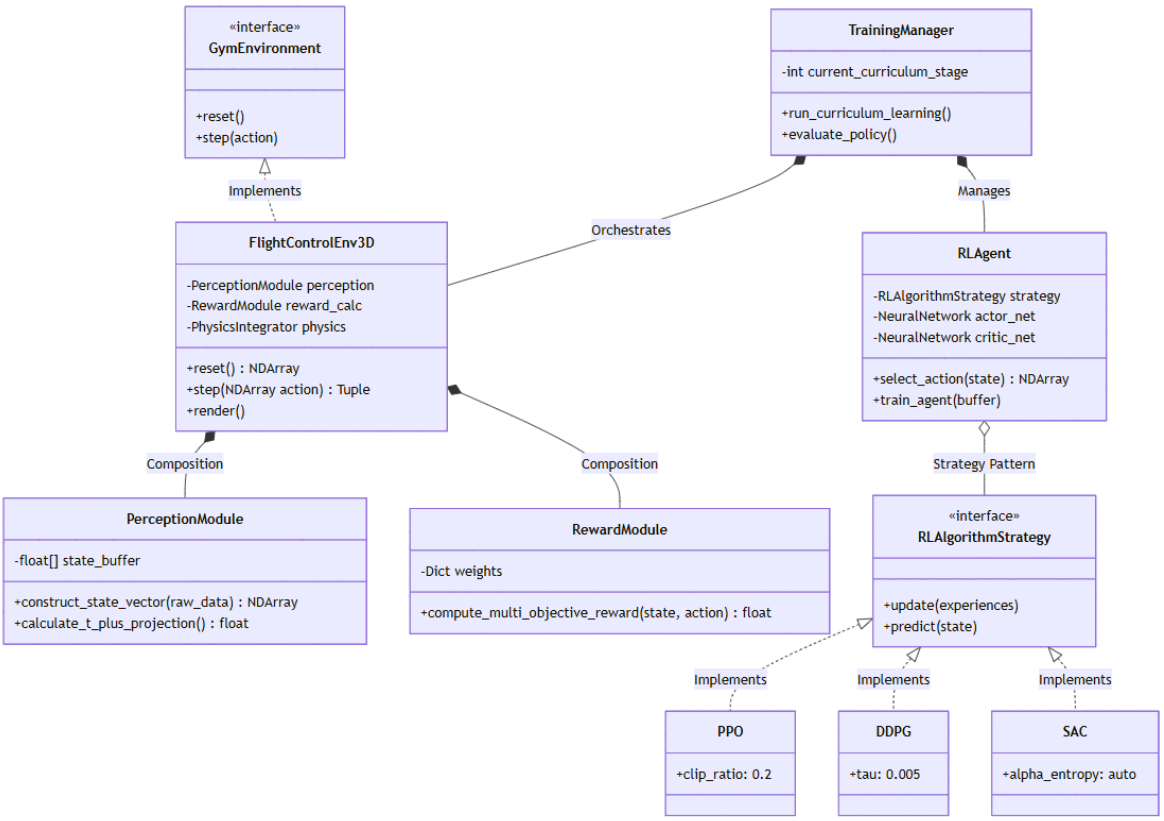


Figure 4.1 : Class Diagram

4.3 State Diagram

The State Diagram maps the dynamic behavioral states of the autonomous helicopter during an active flight episode, heavily emphasizing the proactive safety constraints.

- Following the Initialization and Env Ready states, the helicopter enters FlightOperations, defaulting to TargetTracking.

- **Safety-Critical Interrupts:** The system continuously monitors the environment via sensor scans. If turbulence is detected, it enters WindDisturbance to correct its attitude. More importantly, if the distance to terrain drops below the critical 25-meter threshold, the system forcefully transitions into the ObstacleDetection and ProactiveEvasion states. Here, it calculates an escape vector and executes an evasive maneuver until the safety margin is restored.

- The episode concludes in a TerminalState, which can be a highly rewarded Goal Reached, a heavily penalized Collision Detected, or a Max Steps Reached timeout.

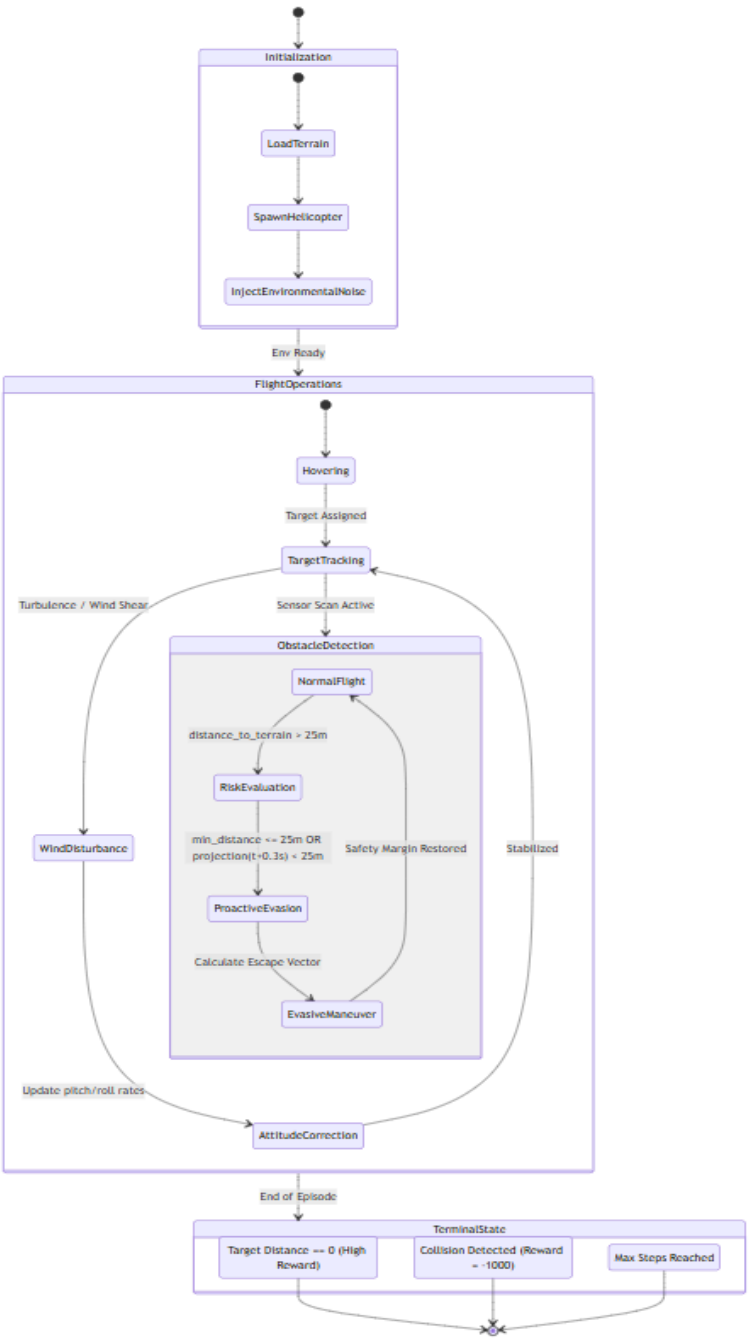


Figure 4.2 : State Diagram

4.4 Sequence Diagram

This sequence diagram details the chronological, step-by-step interaction between system modules during a single Markov Decision Process (MDP) training loop.

- The cycle begins with the Training Manager initializing the environment.
- During the continuous [Every Time Step] loop, the RL Agent sends a continuous action array to the Sim Environment. The physics integrator advances the simulation and yields raw state data.
- This raw data is intercepted by the Perception Module, which constructs and normalizes the state vector (S') before passing it to the agent. Concurrently, the Reward Module evaluates the collision status, distance to target, and fuel efficiency to generate the scalar reward (R).
- The agent stores the (S, A, R, S') transition in its memory buffer. Once the [Episode Done?] condition is met, the loop breaks, and the agent triggers the PPO backpropagation process to update its policy network weights.

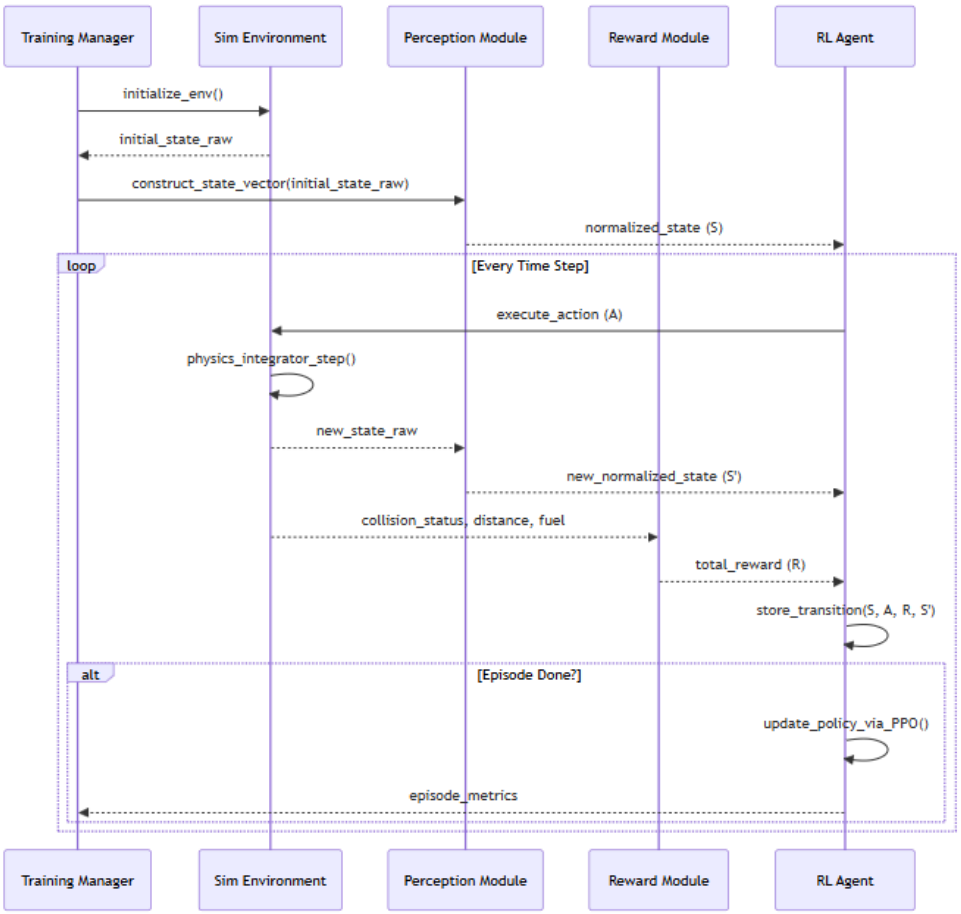


Figure 4.3 : Sequence Diagram

5. DATA DESIGN

The data design maps how the high-frequency physical simulation data is transformed into abstract mathematical tensors for the neural network, stored during training, and logged for post-flight analysis.

5.1 Data Model

5.1.1 Logical Data Model

The Logical Data Model defines the high-level data entities generated and utilized across training scenarios.

- At the top level, a SimulationScenario defines the environment type (e.g., FlightGear or AirSim). Each scenario generates multiple FlightEpisode entities.
- **High-Frequency Episode Data:** Each flight episode executes and logs highly granular time-series data: TelemetryRecord (position, orientation, terrain distance) and AgentAction (roll, pitch, yaw, vertical velocity constraints).
- **Evaluation and Configuration:** The episodes are configured by EnvironmentParams (wind, vibration) and ultimately produce an EvaluationResult. These results are used to tune the AlgorithmConfig and are benchmarked against saved TrainingCheckpoint models, creating a clear lineage from raw flight data to evaluated AI models.

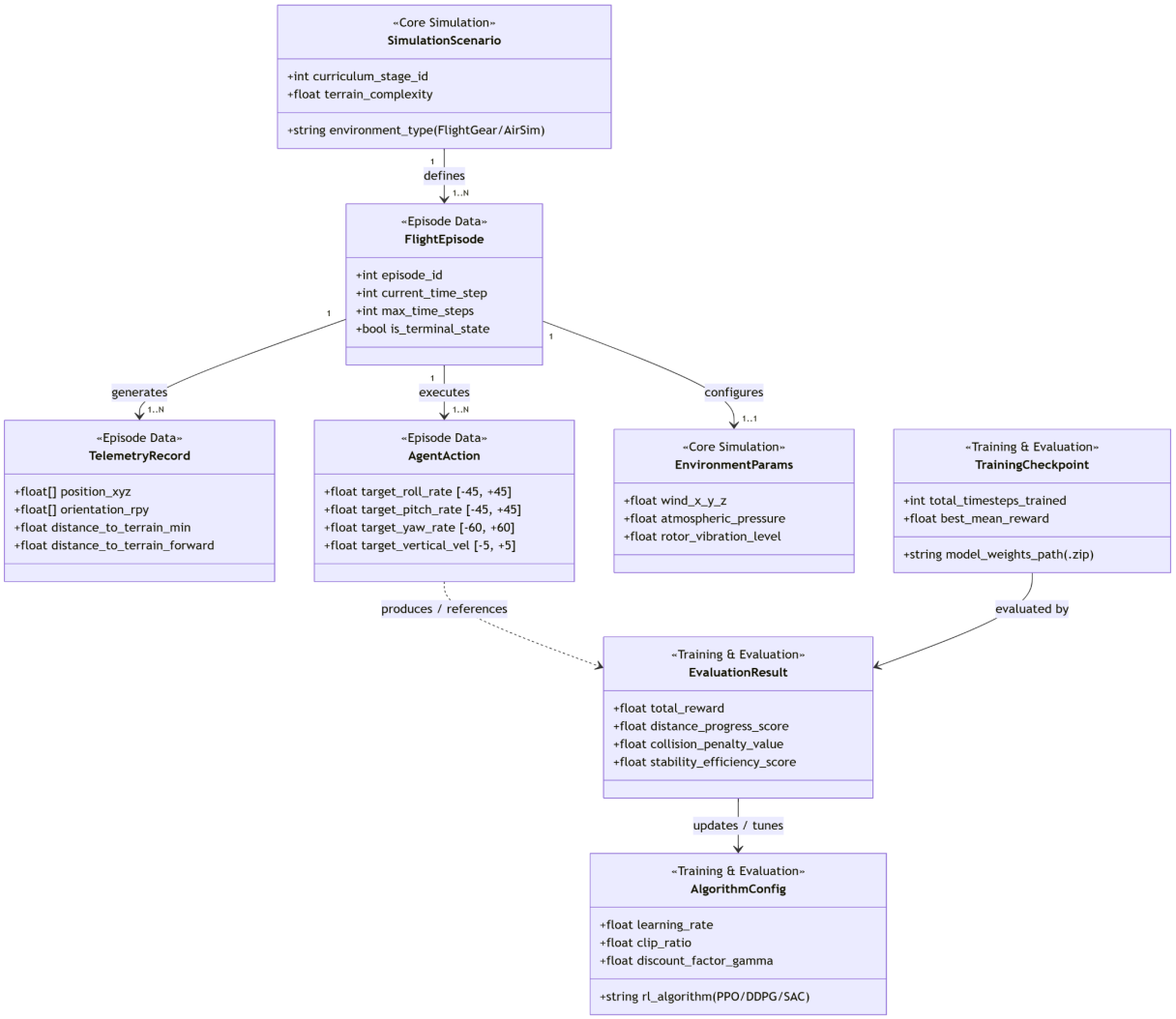


Figure 5.1 : Logical Data Model

5.1.2 Physical Data Model

The Physical Data Model maps how the logical entities are persisted onto the disk storage and file system to support both training efficiency and post-flight analysis.

- **Simulation Data:** Real flight logs (fg_log2.csv) and 3D terrain meshes (.btg) are stored locally and loaded into RAM to feed the simulation engine.
- **Training Monitoring:** As the model trains, loss and reward curves are serialized and written continuously into the /tensorboard_logs/ directory as events.out.tfevents files for real-time visualization.
- **Model Checkpoints:** The Stable-Baselines3 framework persists the trained network weights physically as compressed .zip archives (e.g., ppo_stage4_final.zip). Internally,

these archives contain the precise PyTorch tensors (policy.pth for the Actor, value.pth for the Critic) required to reload and evaluate the agent.

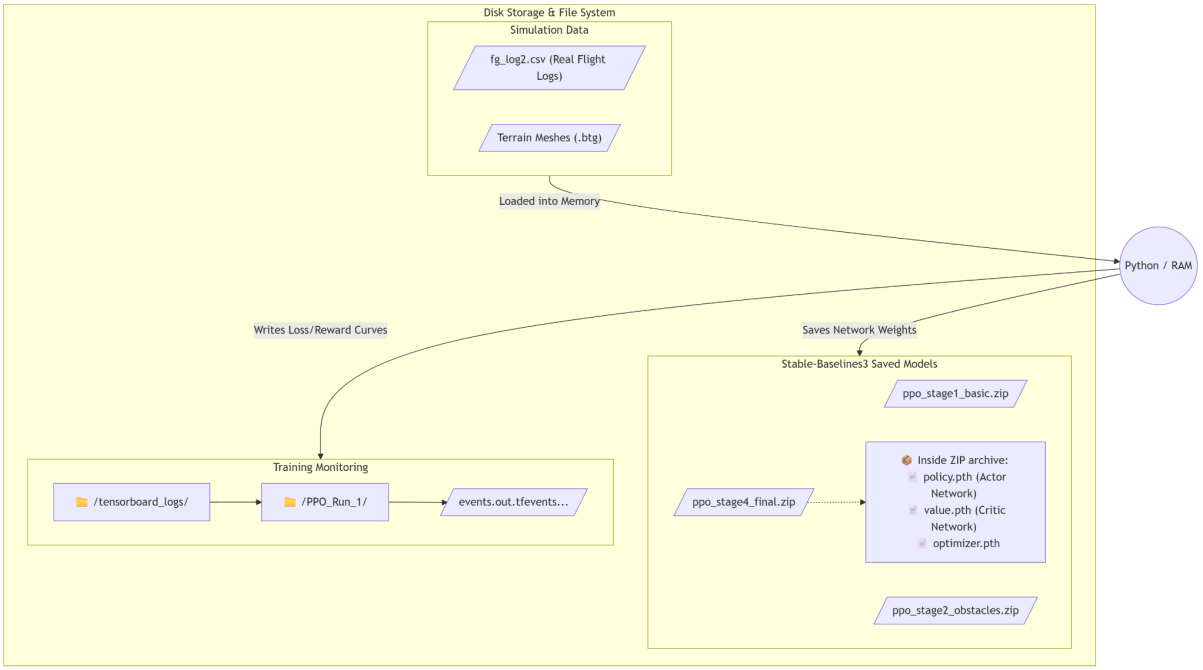


Figure 5.2 : Physical Data Model

5.2 Data Structures

In-memory operations utilize specific Python dataclasses and NumPy arrays to optimize matrix operations necessary for deep learning.

5.2.1 Helicopter State (Observation Space)

The internal HelicopterState object tracks real-time dynamics, which is subsequently flattened into a 1D NumPy array for the neural network:

- *Positional Deltas:* dx, dy, dz (Distance to target)
- *Linear Velocities:* vx, vy, vz
- *Orientation (Euler):* roll, pitch, yaw
- *Angular Rates:* p, q, r
- *Environmental/Sensor:* vibration (float), wind (Tuple of 3 floats), distance_to_terrain metrics.

5.2.2 Action Structure

A 1D NumPy array representing continuous control outputs constrained to specific physical limits:

- [pitch_setpoint, roll_setpoint, yaw_rate_setpoint, throttle].
- Values are bounded (e.g., pitch rate between [-45, +45] degrees/sec) to prevent physically impossible commands.

5.2.3 Reward Components Structure

Rewards are accumulated and tracked individually for granular debugging using a RewardSignal DataClass containing: proximity_reward, collision_penalty, wind_compensation_reward, stability_bonus, and total_reward.

5.2.4 Scenario Configuration Structure

A dictionary-like ScenarioConfig object defines the environment before initialization, dictating: terrain_type, wind_profile, obstacles list, target_position, and episode_time_limit.

5.3 Data Flow

This Level-1 Data Flow Diagram illustrates the continuous, closed-loop pipeline of data transforming across the system's three primary architectural layers.

- **Simulation Physics Layer:** The physical engine (FlightGear/AirSim) acts as the data source, outputting raw IMU, wind, and distance matrices.

- **Perception Layer:** The raw data flows into the Sensor Data Aggregator. Critically, it splits into a Risk Assessment Calculator (to project t+0.3s collisions) and a State Vector Constructor. The output is a highly refined, normalized 20-24 dimensional State Vector (S).

- **Control Logic Layer:** The State Vector feeds into both the RL Policy Network (which outputs the 4-dimensional Action Vector A back to the simulator) and the Reward Computation Module. The resulting transitions (S, A, R, S_{next}) pool into the Experience Replay Buffer. This buffer periodically feeds batches back into the Policy Network to compute gradients via backpropagation, closing the learning loop.

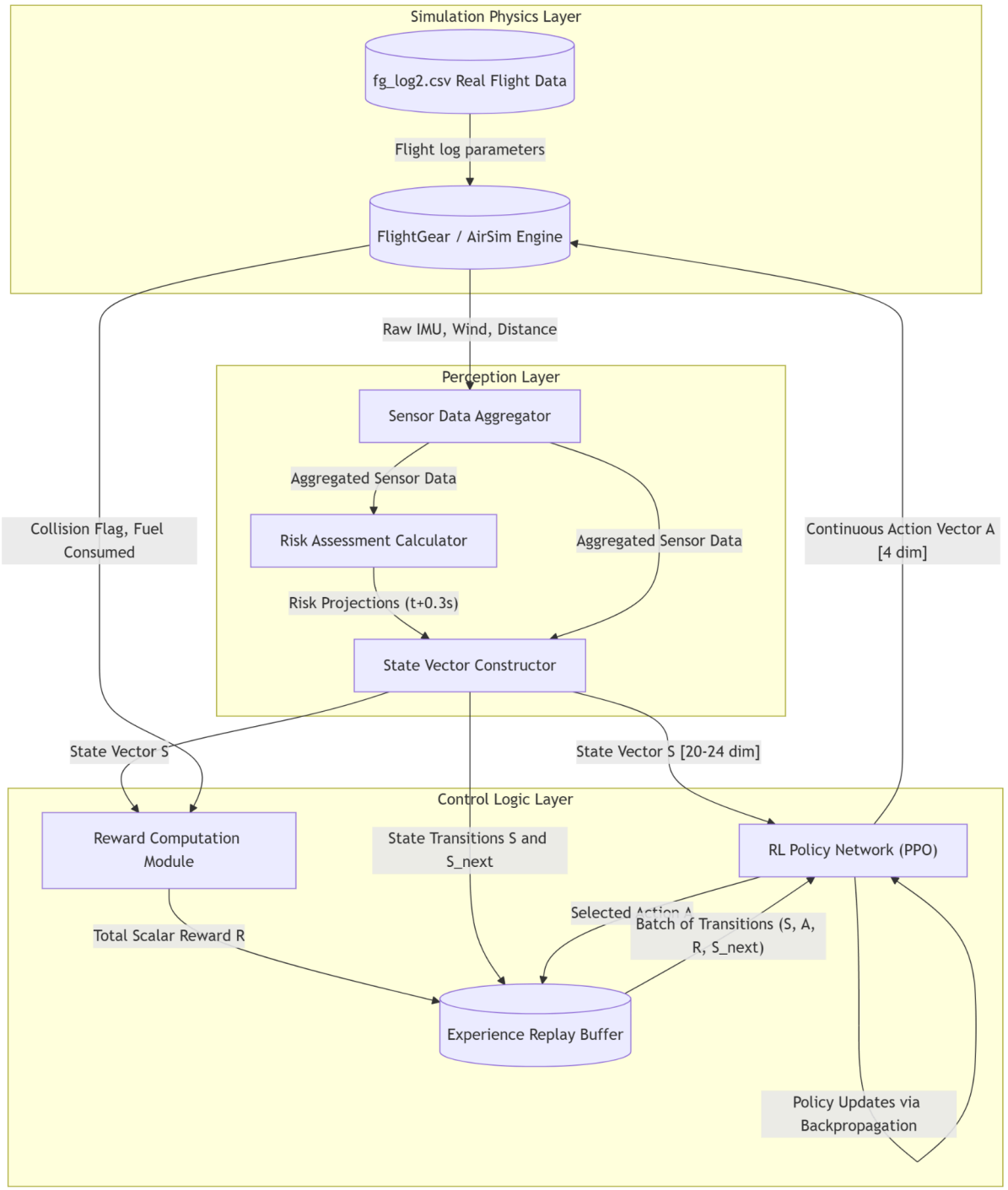


Figure 5.3 : Data Flow Diagram

6. INTERFACE DESIGN

6.1 User Interface (UI)

The user interface of the system is designed to support researchers and developers in managing, monitoring, and evaluating the reinforcement learning (RL) process for autonomous helicopter navigation. Since the system is simulation-based and research-oriented, the interface emphasizes usability, interpretability of results, and flexibility in experimentation.

The UI is structured into three main components: the Training Dashboard, the Scenario Configuration Interface, and the Evaluation Interface. In addition to control functionalities, the interface enables real-time visualization and behavioral analysis, which are critical for understanding the learning process of the RL agent.

6.1.1 Training Dashboard

The Training Dashboard provides real-time monitoring and control over the reinforcement learning process. It allows users to track the progress of the RL agent and evaluate its learning performance across episodes.

The dashboard displays key metrics such as episode number, cumulative reward, and rolling success rate. These metrics help determine whether the training process is converging toward a stable and optimal policy.

In addition, the interface includes real-time visualization of helicopter trajectories, orientation, and environmental factors such as wind vectors. This visualization enables users to intuitively understand how the agent reacts to dynamic and uncertain conditions.

System-level information such as GPU/CPU utilization and learning rate schedules is also presented to monitor computational performance.

Users can interact with the system through controls such as start, pause, and resume training. The interface also provides functionality for saving and loading model checkpoints, ensuring reproducibility and supporting iterative experimentation.

6.1.2 Scenario Configuration Interface (Scenario Builder)

The Scenario Configuration Interface allows users to create and customize simulation environments for training and testing the RL agent. This component is essential for evaluating the robustness and generalization capability of the system.

Users can configure various environmental parameters, including terrain type, obstacle placement, wind conditions, and sensor noise levels. Additionally, the initial position of the helicopter and the target waypoint can be defined.

The interface supports stage-based scenario design, where environments can be simplified (e.g., without obstacles) and progressively made more complex (e.g., including obstacles and environmental disturbances). This approach aligns with the staged learning strategy adopted in the project.

A 3D preview of the configured environment is provided, allowing users to visually verify the scenario setup. The system also includes a validation mechanism to detect infeasible or unsafe configurations.

6.1.3 Evaluation Interface

The Evaluation Interface is designed to analyze and interpret the performance of trained RL models. It provides both quantitative metrics and visual representations of system behavior.

Key performance indicators such as reward trends, success rate, collision rate, time to reach the target, and energy consumption are displayed using graphs and charts. Additionally, stability metrics, including roll and pitch variance, are used to assess the smoothness of flight.

The interface also supports behavioral analysis by comparing simulated flight trajectories with real flight data. This enables validation of the learned policies and helps assess how closely the system mimics realistic helicopter behavior.

Evaluation results can be exported in multiple formats, including PDF for reporting and CSV for further analysis. The interface also supports comparison between different trained models.

6.2 External Interfaces

The system interacts with several external components to enable simulation, learning, and data processing. These interfaces ensure proper integration between system modules and external tools.

6.2.1 Hardware Interfaces

The system does not directly interact with physical hardware. All operations are performed within a software-based simulation environment. Therefore, no hardware-level interfaces such as sensors or actuators are required in the current implementation.

6.2.2 Software Interfaces

The system relies on multiple external software components.

Simulation platforms such as AirSim and FlightGear provide realistic helicopter dynamics, terrain modeling, and environmental conditions. These platforms act as the environment in which the RL agent operates.

The reinforcement learning process is implemented using the Stable-Baselines3 framework, which provides an efficient implementation of the Proximal Policy Optimization (PPO) algorithm.

The system is developed in Python and utilizes several libraries, including NumPy for numerical computations, PyTorch or TensorFlow for neural network implementation, Matplotlib for visualization, and Pandas for data analysis.

6.2.3 Communication Interfaces

Communication between system components is handled through structured interfaces.

Internally, modules communicate through function calls and shared data structures. The interaction between the RL agent and the simulation environment follows the Gymnasium API structure using step() and reset() functions.

If the simulation environment operates externally, communication may be established using network-based protocols such as TCP/IP. This ensures synchronization between the simulation environment and the RL control system.

7. REQUIREMENTS TRACEABILITY

7.1 Requirements Mapping

Requirements traceability establishes a clear relationship between system requirements defined in the Software Requirements Specification (SRS) and the corresponding design components.

Each requirement is mapped to one or more components responsible for its implementation. This ensures that all functional and non-functional requirements are systematically addressed within the system architecture.

For instance, environmental modeling requirements are handled by the simulation environment, while decision-making capabilities are implemented within the RL agent. Similarly, performance evaluation requirements are addressed by the evaluation module.

This mapping improves system transparency, supports validation, and ensures completeness of the design.

Table 7.1 : Functional Requirements Design and Verification Mapping

Req ID	Requirement Description	Design Component(s)	Verification Method
FR1	Simulation Environment Configuration	Simulation Environment Module	Visual inspection of 3D environment; successful initialization of simulator instance.
FR2	Environmental Disturbance Modeling	Simulation Environment Module	Unit tests verifying stochastic generation of wind and pressure noise within defined distributions.
FR3	Multi-Parameter Perception & State Encoding	Perception Module	Unit tests validating data normalization and state vector dimensions ([-1, 1] bounds).
FR4	High-Level Maneuver Command Execution	RL Agent Module & Simulation API	Integration tests verifying network outputs correctly translate to simulation pitch/roll/yaw/throttle.
FR5	Multi-Objective Reward Calculation	Reward Computation Module	Unit tests for each reward sub-component (e.g., negative reward generated on simulated collision).
FR6	Staged Curriculum Learning	Training Manager Module	Smoke testing the training pipeline across multiple difficulty stages without crashes.

FR7	Safe Obstacle Avoidance (25m margin)	Perception Module & Reward Module	Statistical evaluation: tracking minimum distance to obstacles over 100+ evaluation episodes.
FR8	Training Data Logging	Training Manager Module	Verification of generated log files and TensorBoard event files during training.

7.2 Verification Strategy

The verification strategy defines how each requirement is validated to ensure correct system behavior.

Simulation-related requirements are verified through scenario initialization tests and visual inspections. Environmental modeling is validated using statistical analysis to ensure realistic parameter distributions.

State representation is tested by comparing processed observation vectors with raw simulation data. The action system is validated through integration testing to confirm correct mapping between agent outputs and simulation controls.

The reward mechanism is evaluated using scenario-based testing, ensuring that safe behaviors are rewarded and unsafe behaviors are penalized.

In addition to these methods, experimental validation plays a key role in verification. Multiple experiments were conducted to evaluate system behavior under different conditions, including obstacle-based environments, staged learning strategies, and waypoint-based navigation.

Results from these experiments show that initial training with obstacles resulted in unstable behavior, while a staged learning approach significantly improved flight stability and goal-oriented movement. However, waypoint-following performance remains a limitation, indicating the need for further refinement of reward design and training strategies.

Overall, this multi-level verification approach ensures both correctness and robustness of the system.

8. SUPPORTING INFORMATION

8.1 Glossary

This section defines key terms used throughout the document.

- **Reinforcement Learning (RL):** A machine learning paradigm where an agent learns by interacting with an environment through trial and error.
- **Proximal Policy Optimization (PPO):** A reinforcement learning algorithm designed for stable policy updates.
- **State:** A representation of the helicopter's current condition, including position, velocity, and environmental factors.
- **Action:** A set of control commands generated by the RL agent.
- **Reward:** A scalar value that evaluates the effectiveness of an action.
- **Episode:** A complete simulation run from initialization to termination.
- **Telemetry:** Sensor and simulation data collected during execution.
- **Policy Network:** A neural network that determines the agent's actions.
- **Simulation Environment:** A virtual system that models helicopter dynamics and environmental conditions.